

# The store safe, the bag, and two answers

---

## What I built, and one thing I found

---

You asked for all three options plus the safe layer. All three are built. While building them I found a real defect, so I want to lead with that.

Closing a till printed a line that said "**Cash from Register 1 to safe**" — and then booked the money to *Undeposited Funds*, the account that means "sealed in a bag, on its way to the bank." The words and the account disagreed, and the account was wrong. Nothing was ever out of balance, which is why it survived four slices unnoticed. But it meant the books claimed a deposit existed at 4pm, when the bag was not sealed until 9pm. Worse, cash left the vault every morning as float and never came back, so the safe's balance drifted down forever.

Your workflow is now the three layers the big retailers use — **till → safe → bank** — and that is D-77 in the defect log.

## B, explained properly

---

**B is a rule that stops a shift from closing while cash is in limbo.**

Think about the gap between two people. Your employee pulls cash out of the drawer and puts it in a bag. Right then, that money has left the drawer but nobody has counted it into the safe. It belongs to nobody. If it goes missing in that gap, the drawer looks correct, the safe looks correct, and the money is simply gone with no entry anywhere saying so.

So a bag has four states: **available → undeclared → counted → deposited** → back to available. "Undeclared" is the limbo state — cash is in the bag, not yet counted into the safe. Oracle's rule, which I copied, is that *the register cannot close while an undeclared bag exists*. The system stops and says: "**Bag GW-7 was taken out of the drawer but never counted into the safe. Until that count happens the money still belongs to this drawer, so closing now would hide a shortage.**"

That is the whole of B: the shift cannot end until every bag has a name and a number attached to it. The states cannot be skipped and a bag cannot be sealed twice — I tested all sixteen possible moves, not just the four legal ones.

## C, explained properly

---

**C is a mirror, not a rule. It never blocks anything.**

Banks file a report on cash deposits over \$10,000. That report is routine paperwork, it is not trouble, and it is nothing to avoid. The actual crime — structuring — is deliberately keeping deposits *under* that line to stop the report being filed. And here is the part that matters: **purpose is the entire offense**. The IRS manual says plainly that "*accounting entries are not evidence of a structuring violation*," and that examiners must first "*eliminate legitimate reasons for the pattern*." You cannot commit it by accident.

So the check does not refuse anything and does not accuse you. It notices one specific shape — a deposit just under \$10,000 while *more* than \$10,000 is still sitting in the safe — and shows you the same question an examiner would ask. Then you write down the reason at the time: truck broke down, bank closed, short-staffed. If it never comes up, you have lost nothing. If it ever does, you have contemporaneous notes instead of trying to remember a Tuesday from two years ago. A deposit at exactly \$10,000 is not flagged, because that is not under anything.

## Your question 1: over/short on the master vs the sales tills

---

They are genuinely different, and yes, the master till is the one to watch.

A sales till going over or short by a few dollars is ordinary — miscounted change, a fat-fingered entry. It already posts to *Cash Over/(Short)*, and it is self-correcting because the drawer is counted twice a day by two people who both sign for it. What matters is the *pattern*, not the day: small differences in both directions average out and mean nothing, while the same person short the same rough amount every shift means something, and you will now see that.

The master till is different, and this is the real answer to your question. It is shared, it is counted far less often, and it is the one people take cash out of. A shared drawer that only gets counted at shift change has no single person accountable for it, so a shortage there cannot be pinned to anyone. That is why the supply-run answer below matters so much: **most "master till shortages" are not shortages at all — they are cash that walked out with a receipt**. Once those are booked properly, a

genuine master-till variance becomes rare and therefore meaningful.

## Your question 2: the supply run

---

Right now a receipt goes in the drawer and the drawer counts short. A receipt is **evidence, not cash** — so the till really is short, and the ledger should say so out loud with a name attached.

There are two events here, not one, and that is the key.

**When the cash leaves**, it is not an expense — nobody knows yet what was bought, or whether anything was. It is money owed back by a named person. It goes to **12100 Employee Advances Receivable** (an account that already existed, unused). The entry has the employee's name on it and the system refuses to record an unnamed one.

**When the receipt and change come back**, *now* the expense is known, because the receipt says what it was. The expense is booked at the amount the receipt actually says, the change goes back to the safe, and 12100 clears to zero.

Booking it as an expense when the cash leaves would be wrong twice: it records a purchase that has not happened, at an amount that changes when the change comes back — and nothing ever asks for the receipt, because no account is left holding anyone's name. With 12100, anything still sitting there is a live question: *who has our money, and where is the receipt?* If someone hands back more receipt than cash, the system refuses rather than quietly flipping the sign, because that is money **you owe them**, not change owed to the till.

## Proof and what is not done

---

All six gates green. **12,858 tests pass, up from 12,824 — 34 new**. I broke the new code twenty different ways on purpose; **all 20 were caught**, including two that strip the bag number off the entry.

Being straight about the gap: the builders and the `deposit_bags` table are finished and tested, but **no screen calls them yet** — you cannot seal a bag from the app today. That is recorded honestly as unreachable in the census rather than quietly counted as done. The screens are the next slice.